



Машинное обучение

Лекция 09:

Логистическая регрессия и
оценки качества классификации



Что рассмотрим сегодня?

- Логистическая функция
- Постановка задачи логистической регрессии
- Минимизация кросс-энтропии
- Свойства логистической функции
- Градиентный спуск
- Переобучение и регуляризация
- Линейно неразделимые случаи
- Многоклассовая классификация
- Оценка качества модели

Логистическая функция



Логистическая функция

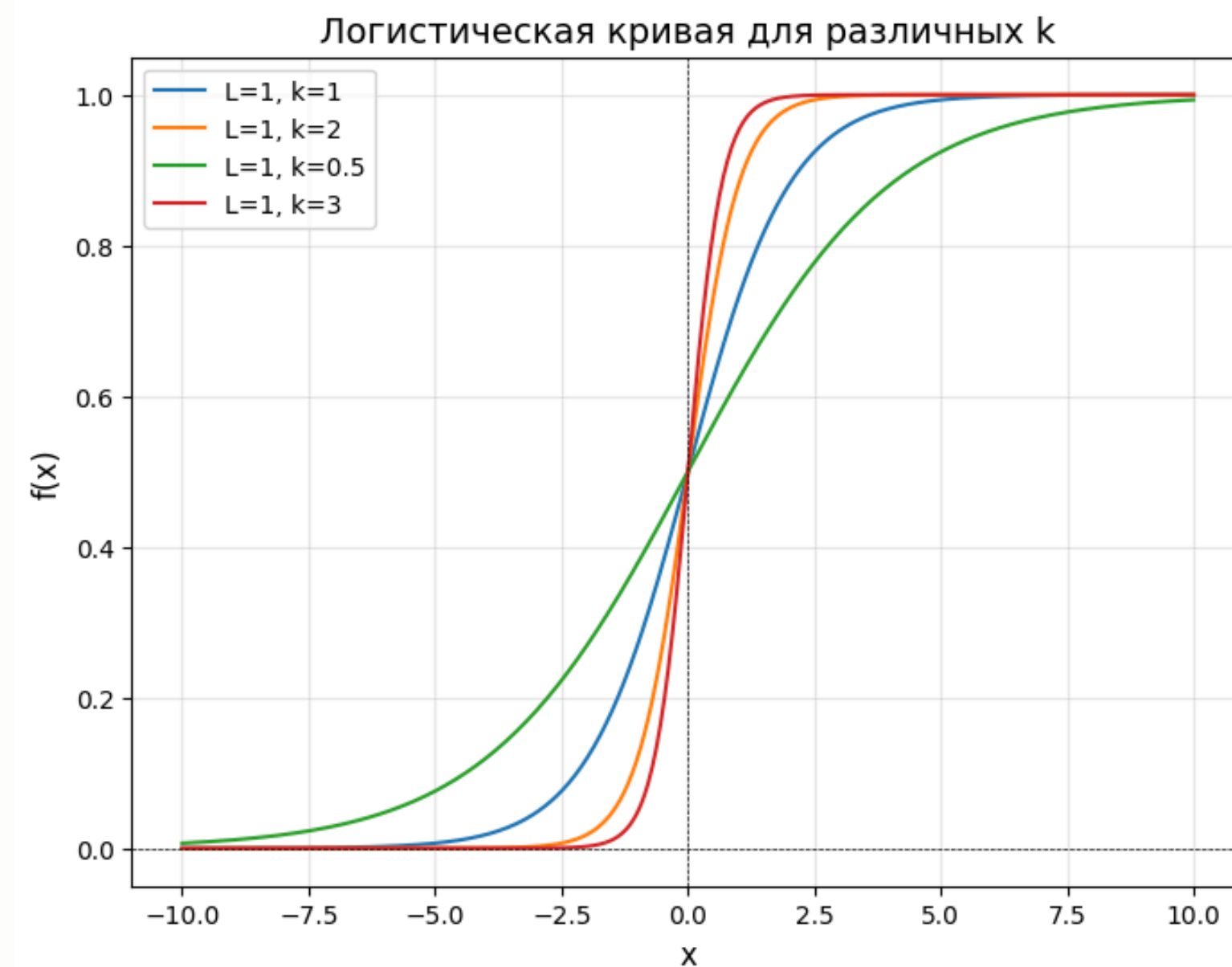
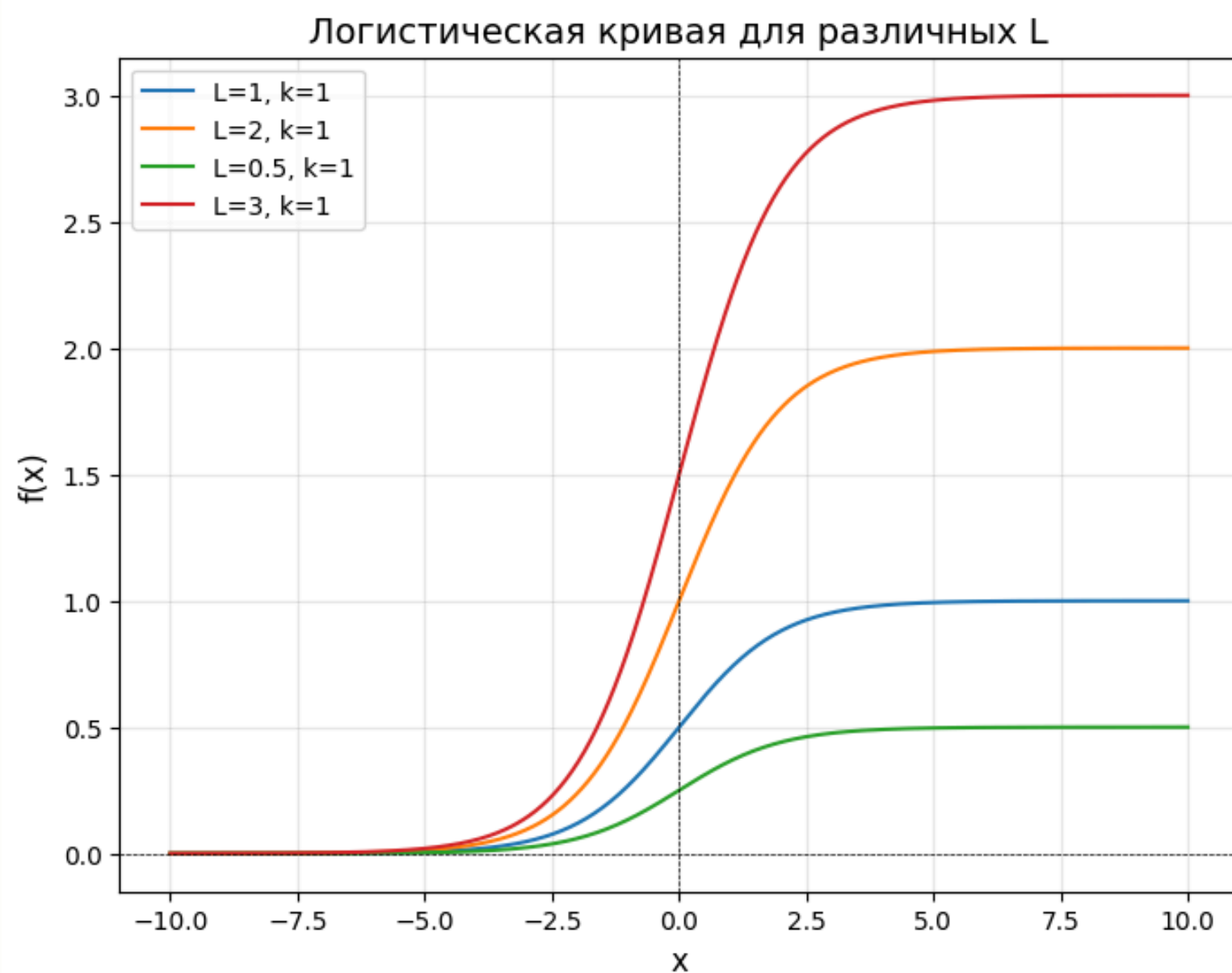
Логистическая функция или **логистическая кривая** – это S-образная кривая (сигмоидальная кривая), заданная уравнением

$$f(x) = \frac{L}{1 + e^{-kx}}, \quad x \in \mathbb{R},$$

где k отвечает за кривизну кривой (логистическую скорость роста), а $L = \sup f(x)$; в свою очередь $\inf f(x) = 0$.



Логистическая функция



Постановка задачи логистической регрессии



Постановка задачи

Пусть задана выборка $\mathbb{D} = (X|y)_{i=1}^n$, где $X \subseteq \mathbb{X} = \mathbb{R}^{n \times m}$, $y \subseteq \mathbb{Y} = \{0, 1\}$, то есть

$$\mathbb{D} = \begin{pmatrix} x_{11} & \dots & x_{1m} & y_1 \\ \vdots & \ddots & \vdots & \vdots \\ x_{n1} & \dots & x_{nm} & y_n \end{pmatrix}.$$

Необходимо найти такую функцию $f : \mathbb{X} \rightarrow [0, 1]$, которая оценивает для объекта $x \in \mathbb{R}^m$ вероятность принадлежности к классу $y = 1$.



Постановка задачи

У нас есть логистическая функция $\sigma : \mathbb{R} \rightarrow [0, 1]$, заданная в виде

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

для которой

- при $z \rightarrow +\infty$ следует $\sigma(z) = 1$;
- при $z \rightarrow -\infty$ следует $\sigma(z) = 0$;
- при $z = 0$ следует $\sigma(z) = 0.5$.

То есть ее можно использовать как значение вероятности.



Постановка задачи

А с помощью модели линейной регрессии мы научились строить отображение $f : \mathbb{X} \rightarrow \mathbb{R}$. Введем параметр $w = (w_0, \dots, w_m) \in \mathbb{W} = \mathbb{R}^m$ и добавим фиктивный столбец $x_0 = (1, \dots, 1)^T \in \mathbb{R}^n$ в X . Тогда модель линейной регрессии мы обозначали

$$f(x_i, w) = x_i^T w, \quad i = 1, \dots, n,$$

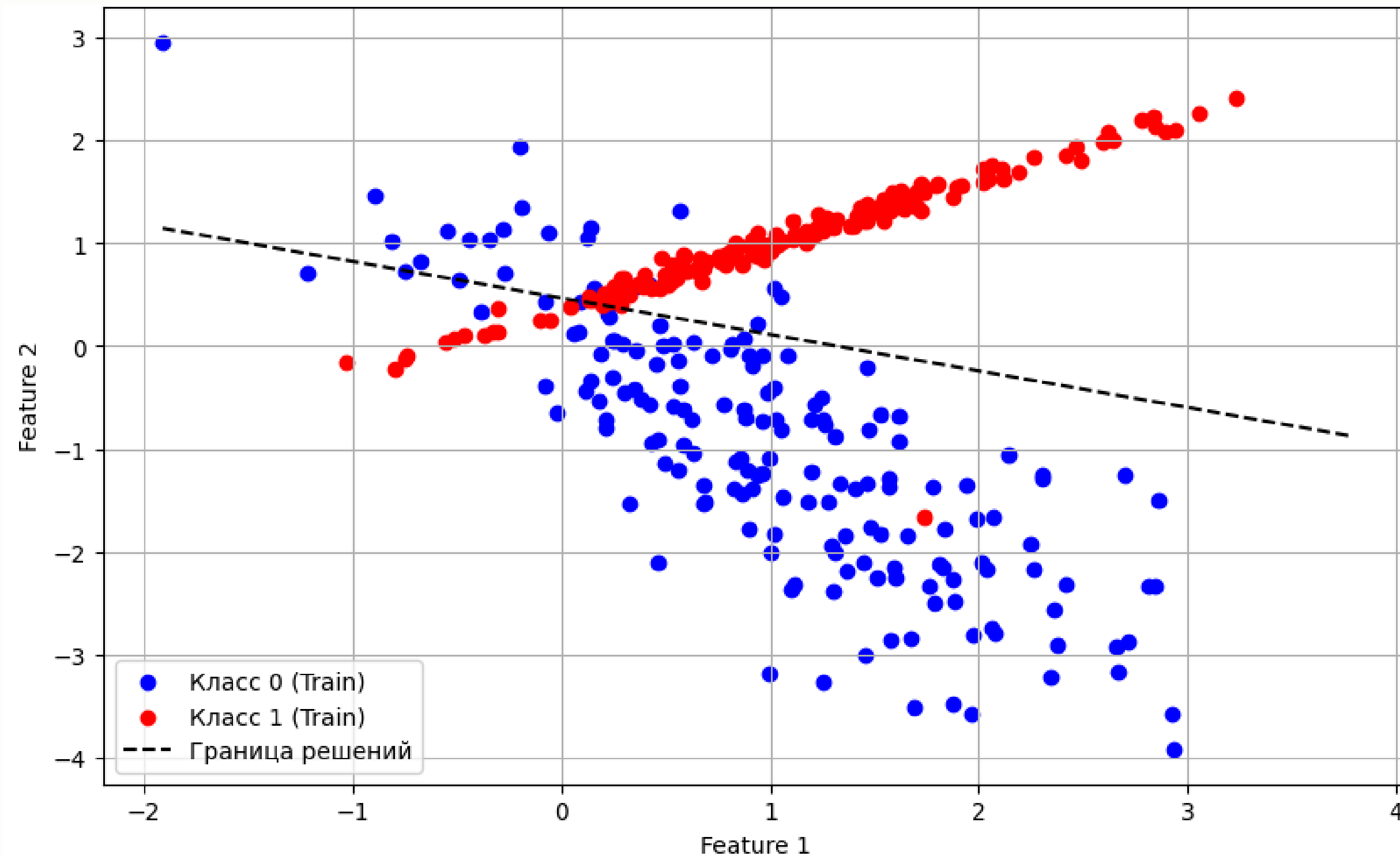
при этом важно, чтобы между признаками не было мультиколлинеарности.

В итоге, подставляя линейную регрессию в логистическую функцию мы сможем определить вероятность положительной метки

$$P(y_i = 1|x_i) = \sigma(x_i^T w) = \frac{1}{1 + e^{-x_i^T w}}, \quad i = 1, \dots, n.$$

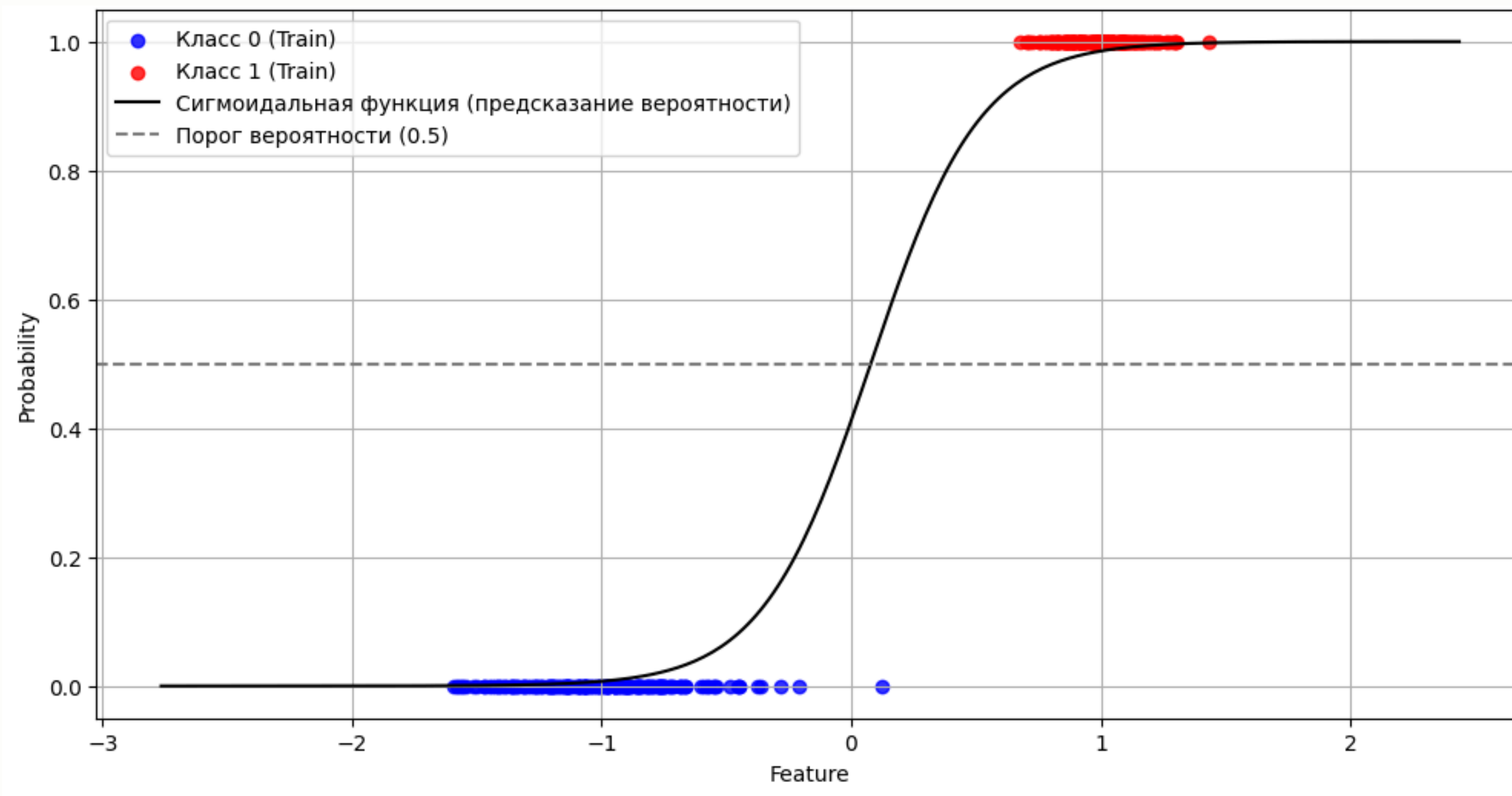


Постановка задачи





Постановка задачи



Минимизация кросс-энтропии



Функция правдоподобия

Пусть случайная величина y_i , соответствующая метке класса для объекта $x_i \in \mathbb{R}^{m+1}$ имеет распределение Бернулли с параметром $p = \sigma(x_i^T w)$, то есть

	$y_i = 0$	$y_i = 1$
$P(y_i = k)$	$1 - \sigma(x_i^T w)$	$\sigma(x_i^T w)$

Тогда плотность распределения случайной величины y_i можно записать следующим образом

$$p(y_i|x_i, w) = \sigma(x_i^T w)^{y_i} \cdot (1 - \sigma(x_i^T w))^{1-y_i}.$$

Для отыскания оценки параметра $\hat{w}$ воспользуемся методом максимального правдоподобия, так как мы снова имеем дело со случайной выборкой. По методу максимального правдоподобия составим функцию правдоподобия

$$L(X, y, w) = \prod_{i=1}^n p(y_i|x_i, w) = \prod_{i=1}^n \sigma(x_i^T w)^{y_i} \cdot (1 - \sigma(x_i^T w))^{1-y_i} \rightarrow \max_w.$$



Кросс-энтропия

$$L(X, y, w) = \prod_{i=1}^n p(y_i | x_i, w) = \prod_{i=1}^n \sigma(x_i^T w)^{y_i} \cdot (1 - \sigma(x_i^T w))^{1-y_i} \rightarrow \max_w.$$

Снова чтобы задачу максимизации было проще решать, возьмем логарифм правдоподобия

$$\ln L(X, y, w) = \sum_{i=1}^n y_i \cdot \ln(\sigma(x_i^T w)) + (1 - y_i) \cdot \ln(1 - \sigma(x_i^T w)) \rightarrow \max_w.$$

Обычно получившуюся функцию принято домножать на (-1)

$$H(y, \sigma(x_i^T w)) = -\ln L(X, y, w) = -\sum_{i=1}^n \left(y_i \cdot \ln(\sigma(x_i^T w)) + (1 - y_i) \cdot \ln(1 - \sigma(x_i^T w)) \right) \rightarrow \min_w$$

и называть **кросс-энтропией**. Отсюда и метод обучения – **минимизация кросс-энтропии**. Таким образом, мы получили функцию потерь для логистической регрессии.



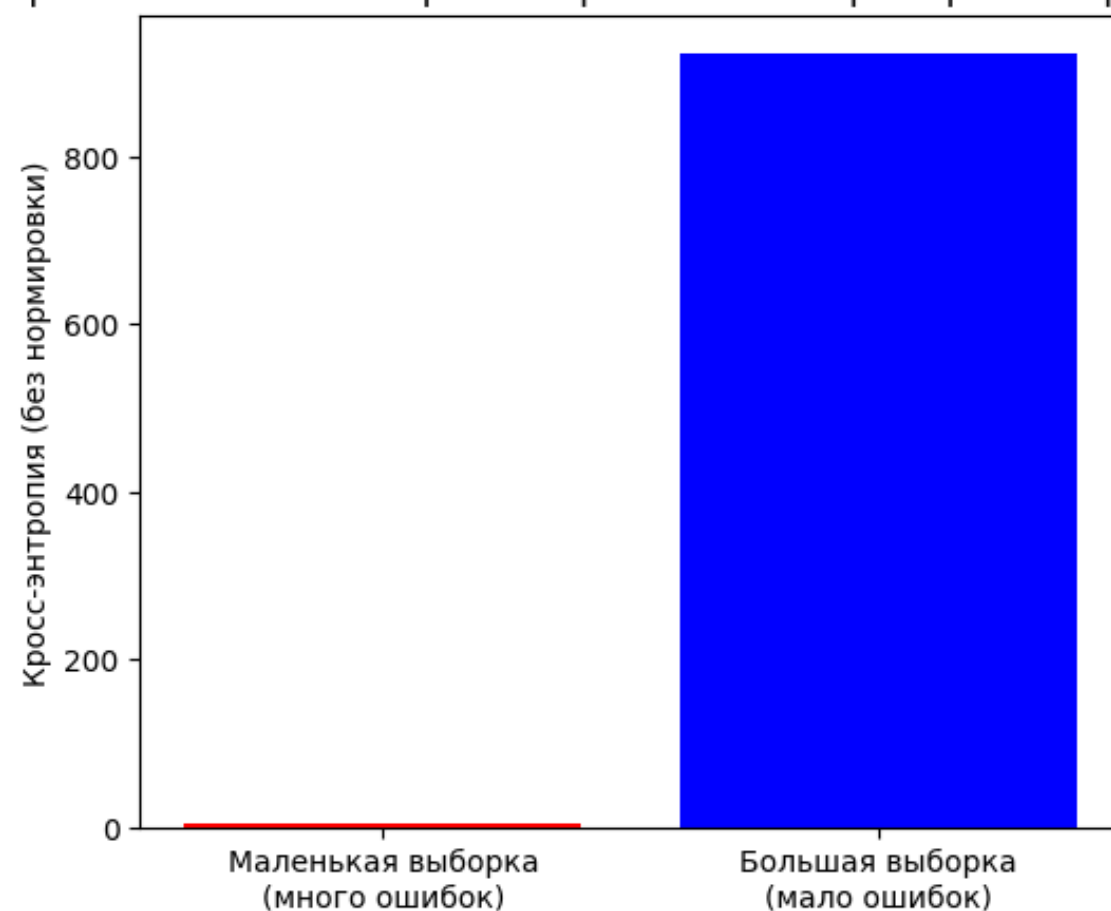
Нормировка

По аналогии с MSE, чтобы сравнивать модели на выборках разных размеров, функцию кросс-энтропии лучше нормировать

$$\frac{H(y, \sigma(x_i^T w))}{n} = -\frac{1}{n} \sum_{i=1}^n \left(y_i \cdot \ln(\sigma(x_i^T w)) + (1 - y_i) \cdot \ln(1 - \sigma(x_i^T w)) \right) \rightarrow \min_w$$

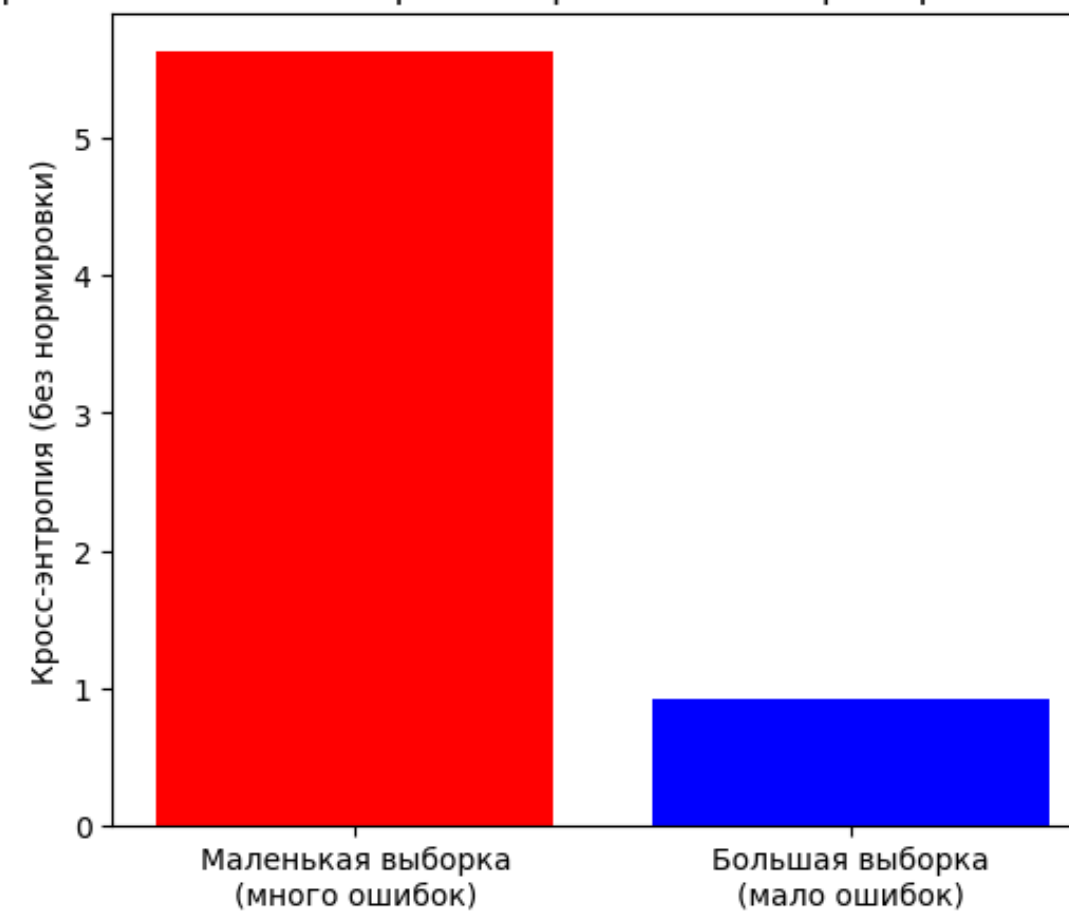
Кросс-энтропия на маленькой выборке (4 объекта, много ошибок): 5.6268
Кросс-энтропия на большой выборке (1000 объектов, мало ошибок): 921.1536

Сравнение значений кросс-энтропии на выборках разного размера



Кросс-энтропия на маленькой выборке (4 объекта, много ошибок): 5.6268
Кросс-энтропия на большой выборке (1000 объектов, мало ошибок): 0.9212

Сравнение значений кросс-энтропии на выборках разного размера



Свойства логистической функции



Свойства

Свойства логистической функции

1. $1 - \sigma(z) = \sigma(-z),$

$$1 - \sigma(z) = 1 - \frac{1}{1 + e^{-z}} = \frac{1 + e^{-z} - 1}{1 + e^{-z}} = \frac{e^{-z}}{1 + e^{-z}} = \frac{1}{e^z + 1} = \sigma(-z)$$

2. $\sigma'(z) = \sigma(z)(1 - \sigma(z)),$

$$\frac{d\sigma(z)}{dz} = \frac{d}{dz} \left(\frac{1}{1 + e^{-z}} \right) = -\frac{(-1) \cdot e^{-z}}{(1 + e^{-z})^2} = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z)(1 - \sigma(z)).$$

Градиентный спуск



Градиентный спуск (вспомним)

$$L(f, X, y) = \sum_{i=1}^n \mathcal{L}(f(x_i, w), y_i) \rightarrow \min_w$$

Вектор частных производных (градиент) по параметрам в общем виде:

$$\frac{\partial L}{\partial w_j} = \sum_{i=1}^n \frac{\partial \mathcal{L}}{\partial f} \cdot \frac{\partial f}{\partial w_j}.$$

Градиент функции в точке направлен в сторону её наискорейшего роста, а антиградиент (противоположный градиенту вектор) в сторону наискорейшего убывания. То есть имея какое-то приближение оптимального значения параметра $w^{(k)}$, мы можем его улучшить, посчитав градиент функции потерь в точке и немного сдвинув вектор весов в направлении антиградиента:

$$w_j^{(k+1)} = w_j^{(k)} - \alpha \frac{\partial L}{\partial w_j^{(k)}},$$

где параметр $\alpha > 0$ называют темпом обучения (learning rate).



Градиент кросс-энтропии

Для решения задачи минимизации кросс-энтропии не принято выводить аналитическую формулу. Обычно сразу же используется метод градиентного спуска. Сперва вычислим, чему равен градиент кросс-энтропии (без нормирующего множителя). Поскольку, используя свойство $1 - \sigma(z) = \sigma(-z)$,

$$H(y, \sigma(Xw)) = - \sum_{i=1}^n \left(y_i \cdot \ln(\sigma(x_i^T w)) + (1 - y_i) \cdot \ln(\sigma(-x_i^T w)) \right),$$

то, в силу того, что $\sigma'(z) = \sigma(z)\sigma(-z)$,

$$\begin{aligned} \frac{\partial H(y, \sigma(Xw))}{\partial w} &= \\ &= - \sum_{i=1}^n \left(y_i \cdot \frac{x_i^T}{\sigma(x_i^T w)} \cdot \sigma(x_i^T w) \cdot \sigma(-x_i^T w) + (1 - y_i) \cdot \frac{(-x_i^T)}{\sigma(-x_i^T w)} \cdot \sigma(-x_i^T w) \cdot \sigma(x_i^T w) \right) = \\ &= - \sum_{i=1}^n x_i^T \left(y_i \cdot (1 - \sigma(x_i^T w)) - (1 - y_i) \cdot \sigma(x_i^T w) \right) = - \sum_{i=1}^n x_i^T (y_i - \sigma(x_i^T w)). \end{aligned}$$



Алгоритм градиентного спуска

А тогда, если записать это в векторном виде с нормирующим множителем, то

$$\frac{\partial H(y, \sigma(Xw))}{\partial w} = -\frac{1}{n}X^T(y - \sigma(Xw)),$$

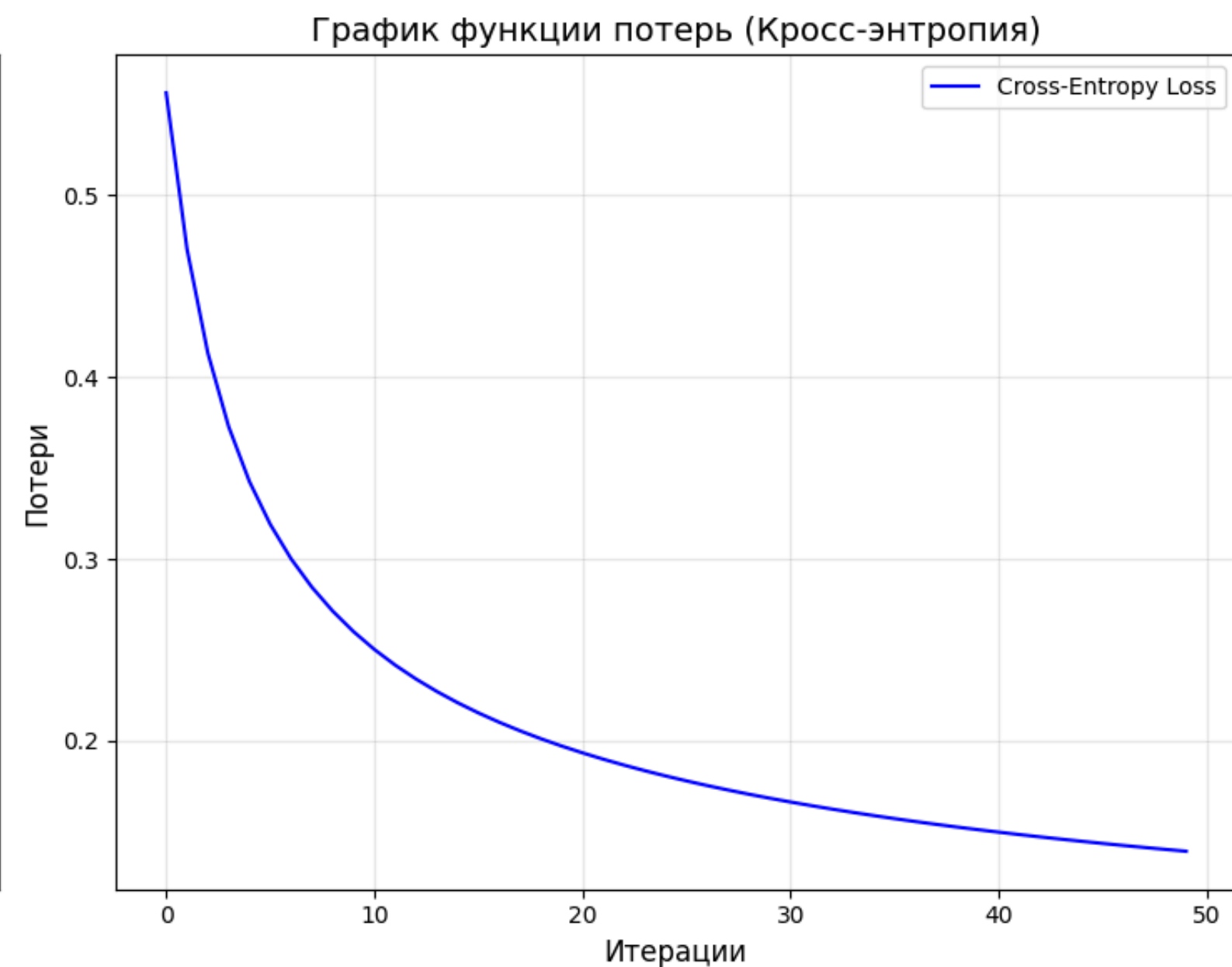
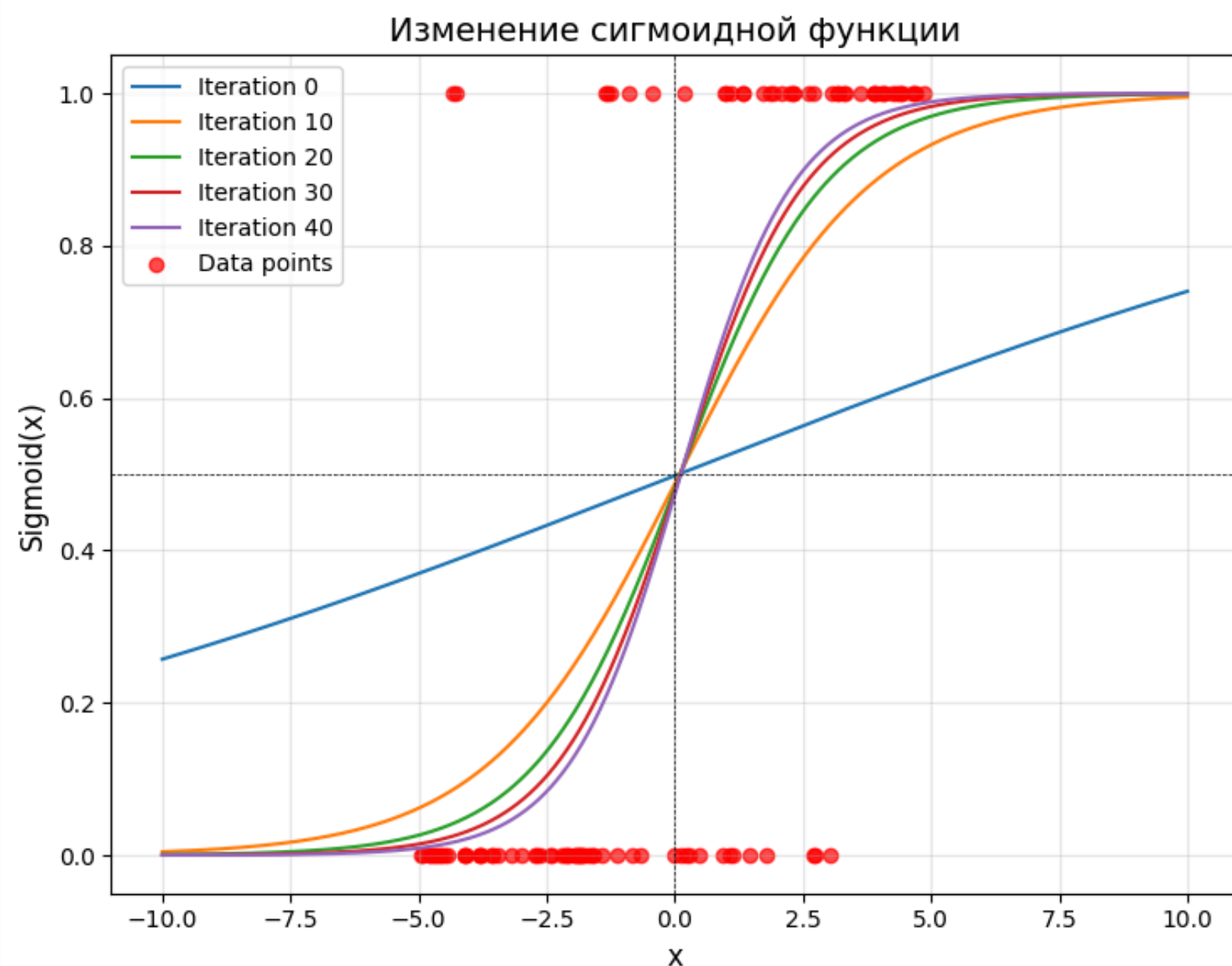
что достаточно схоже с градиентом для линейной регрессии. Таким образом, градиентный спуск для логистической регрессии задается в виде формулы

$$w^{(k+1)} = w^{(k)} - \alpha \cdot \frac{1}{n}X^T(\sigma(Xw^{(k)}) - y), \quad k = 0, 1, \dots,$$

где $\alpha > 0$.

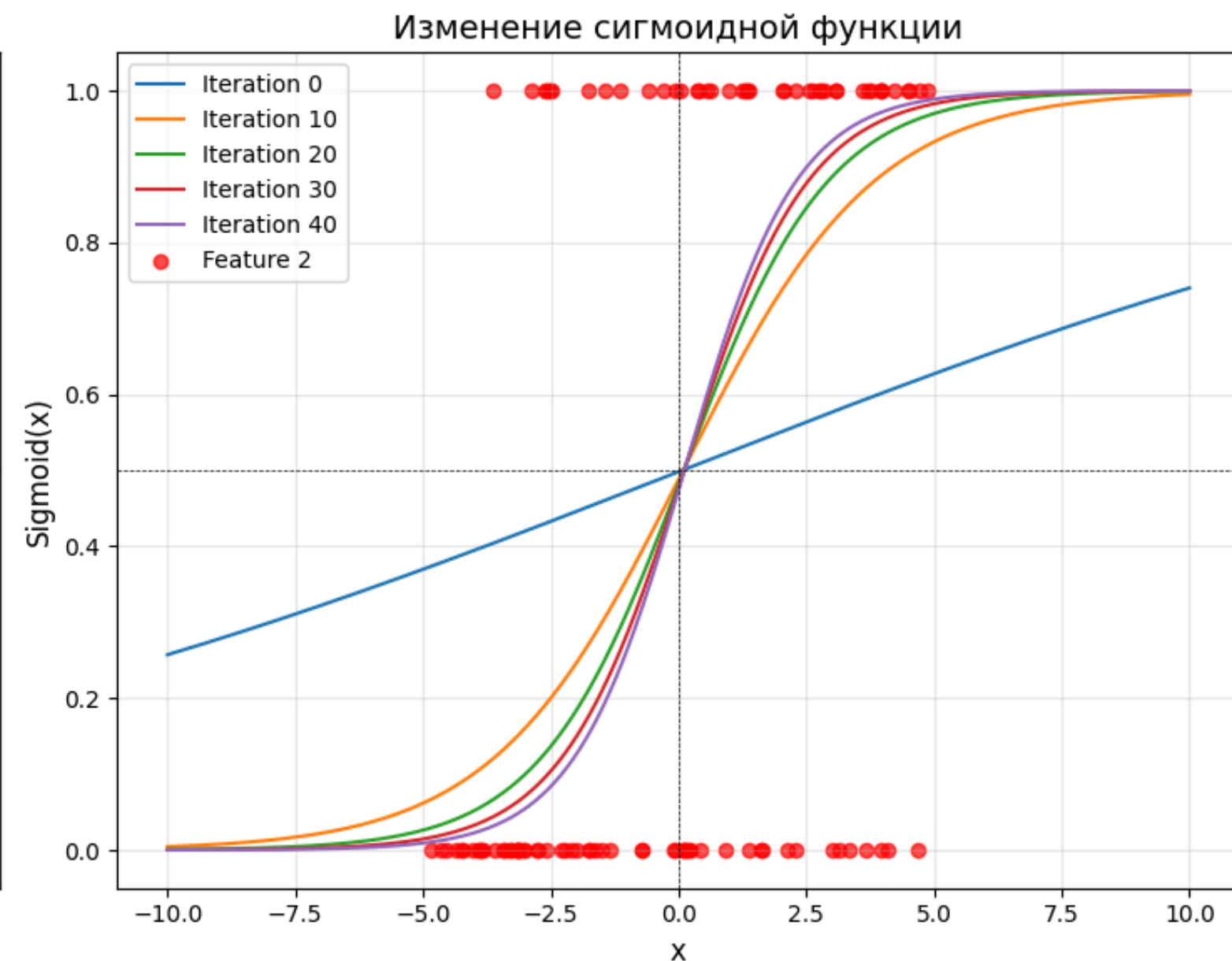
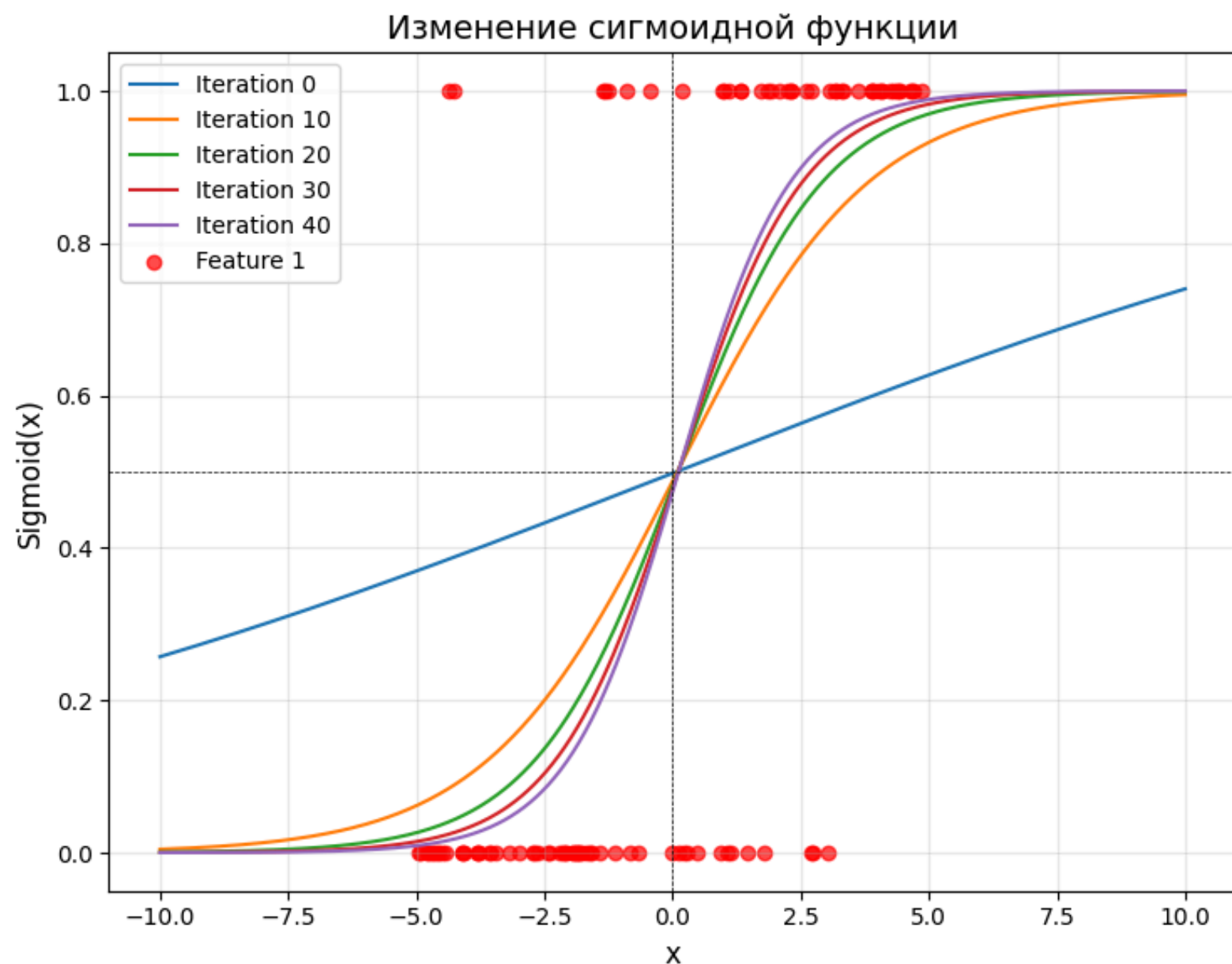


Алгоритм градиентного спуска



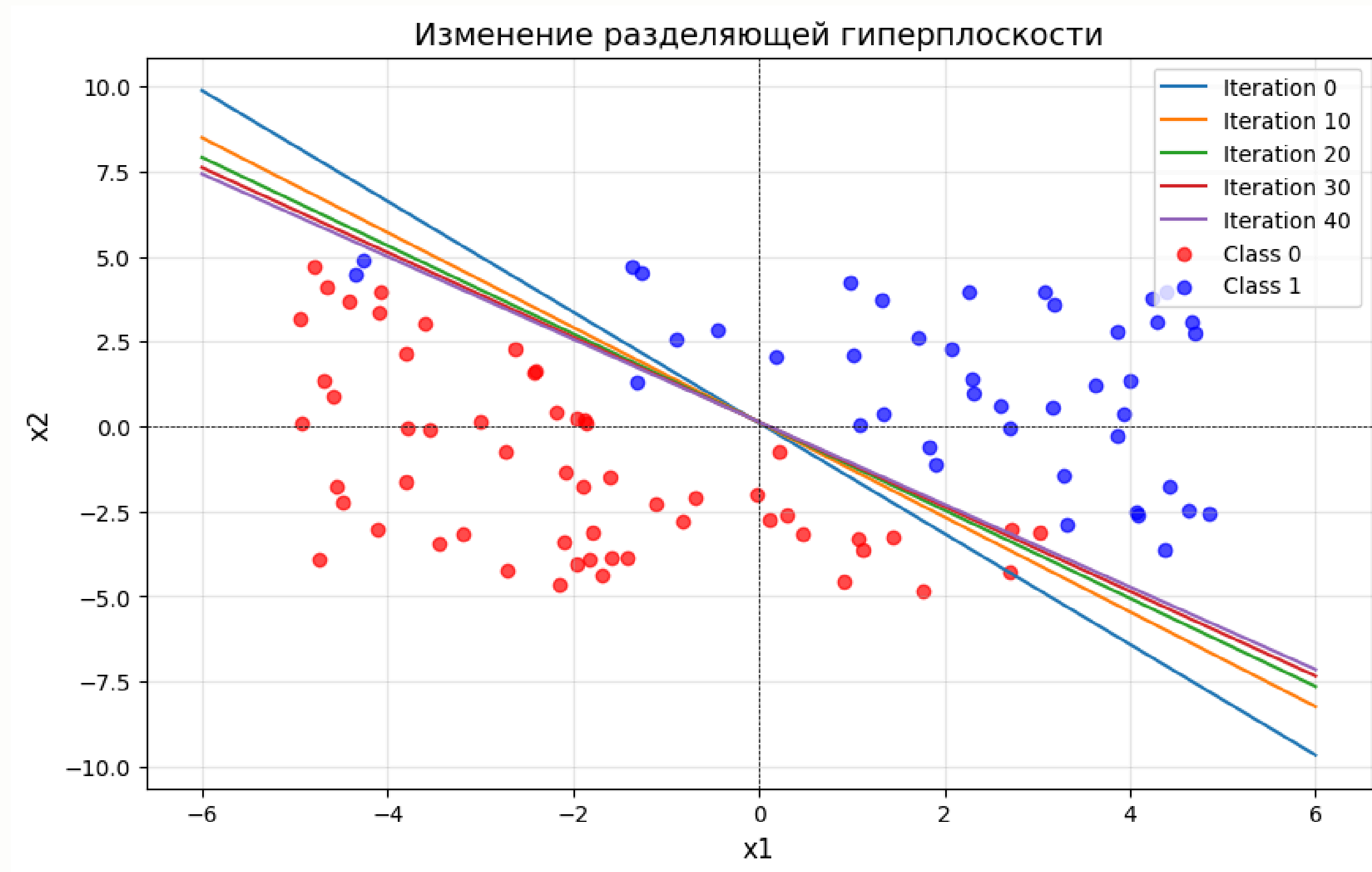


Алгоритм градиентного спуска





Алгоритм градиентного спуска



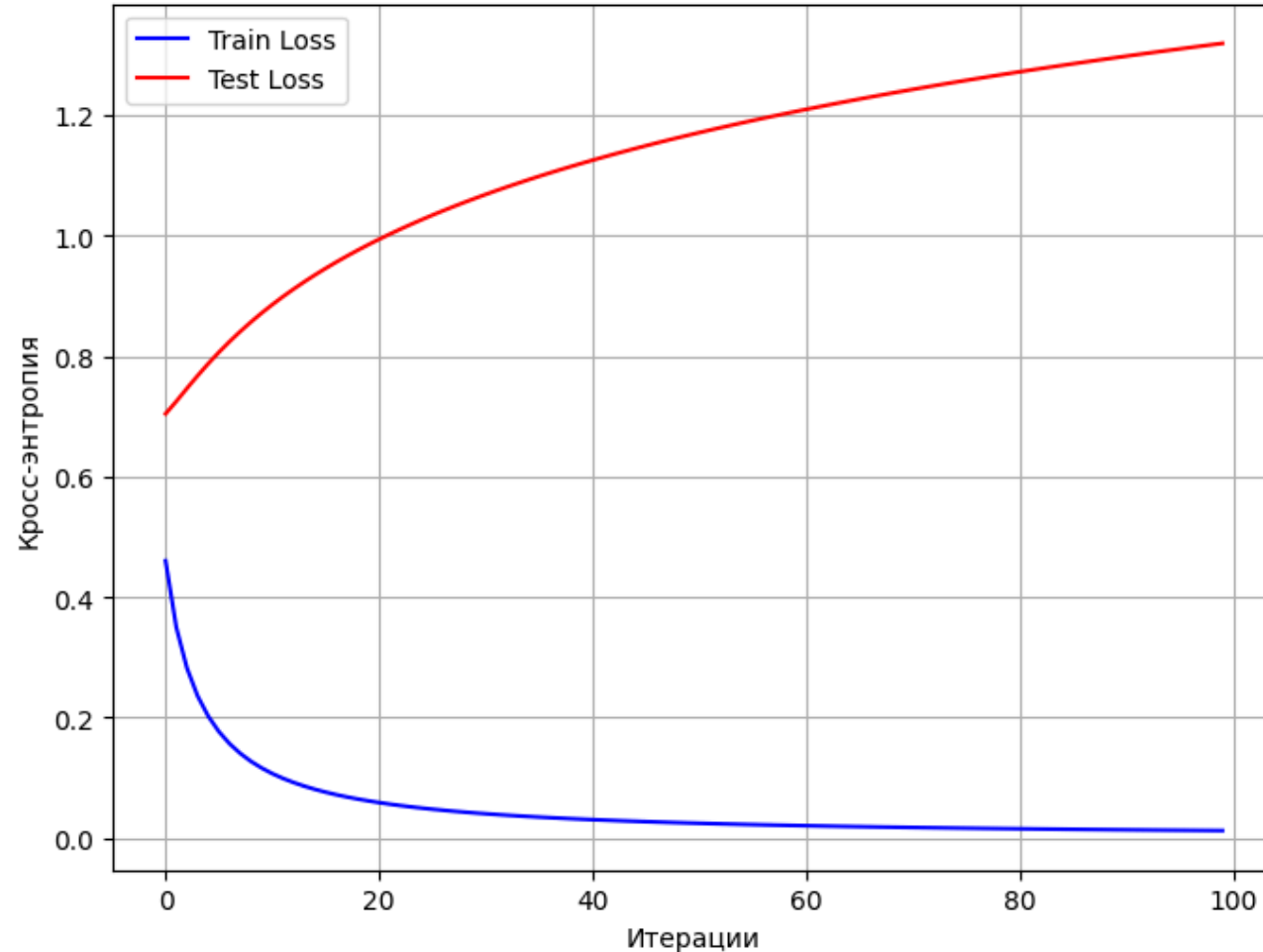
Переобучение и регуляризация



Переобучение

Как и модели линейной регрессии, логистической регрессии тоже свойственно переобучаться. Например, это может происходить, когда признаков сильно больше, чем объектов в обучающей выборке, то есть $m \gg n$. В этом случае модель может хорошо подстроиться под небольшое число элементов в обучающей выборке (за счет большого числа параметров), при этом ее обобщающая способность будет плохой. Например, пусть в обучающей выборке $n = 20$, $m = 50$. Тогда

Переобучение логистической регрессии: потери на обучающих и тестовых данных





Регуляризация

Для борьбы с переобучением в логистической регрессии также можно добавлять регуляризационное слагаемое. Причем в зависимости от выбранной нормы разделяют

- Ridge (или L2) регуляризацию

$$\mathcal{L}_\lambda(X, y, w) = -\frac{1}{n} \sum_{i=1}^n \left(y_i \cdot \ln(\sigma(x_i^T w)) + (1 - y_i) \cdot \ln(1 - \sigma(x_i^T w)) \right) + \lambda \sum_{j=0}^m w_j^2;$$

- LASSO (или L1) регуляризацию

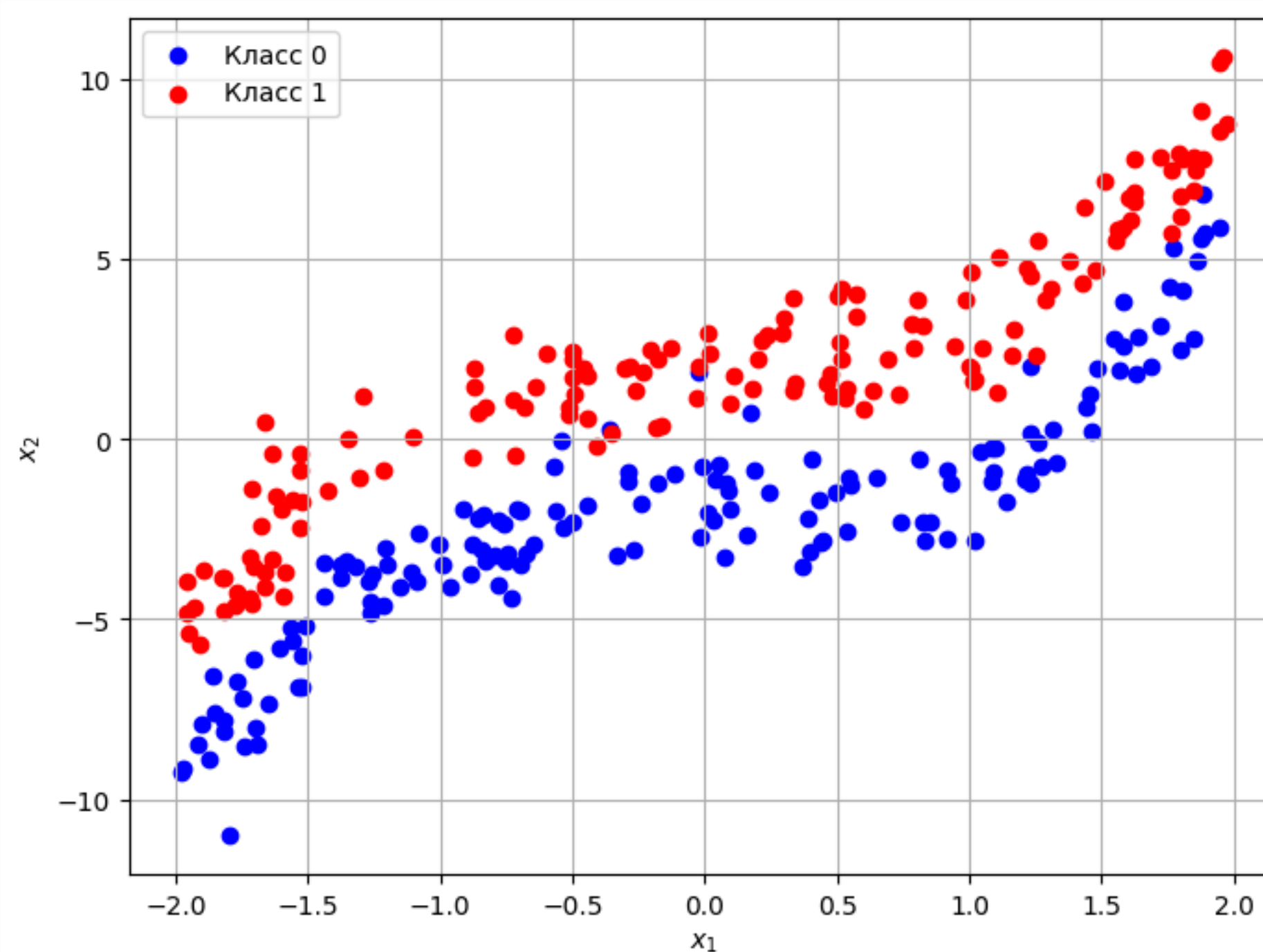
$$\mathcal{L}_\lambda(X, y, w) = -\frac{1}{n} \sum_{i=1}^n \left(y_i \cdot \ln(\sigma(x_i^T w)) + (1 - y_i) \cdot \ln(1 - \sigma(x_i^T w)) \right) + \lambda \sum_{j=0}^m |w_j|.$$

Вся идея регуляризации аналогична модели линейной регрессии: то есть вычисляется градиент $\frac{\partial \mathcal{L}_\lambda(X, y, w)}{\partial w}$, а затем производится минимизация функции потерь $\mathcal{L}_\lambda(X, y, w)$ по методу градиентного спуска.

Случай линейно неразделимой выборки

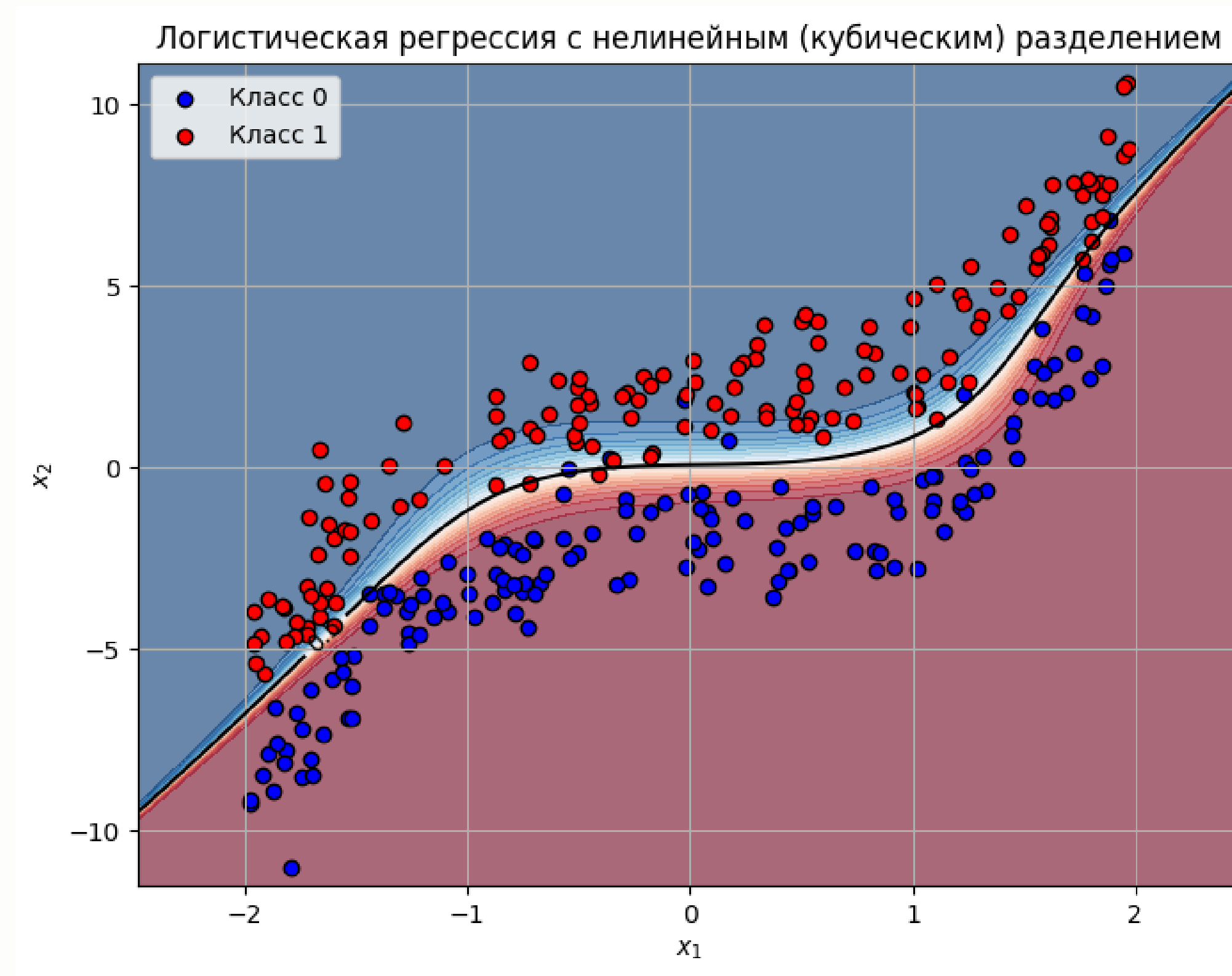


Линейно неразделимая выборка





Кубическая аппроксимация





Кубическая аппроксимация

По аналогии с линейной регрессией мы перешли из исходного признакового пространства в новое пространство

$$X = \{x_1, x_2\} \xrightarrow{g} X' = \{x'_1, x'_2, x'_3, x'_4, x'_5, x'_6\},$$

где

$$g(x) = (x_1, x_2, x_1^2, x_2^2, x_1^3, x_2^3)$$

Тогда, имея базовую модель

$$\sigma(X, w) = \frac{1}{1 + \exp(-(w_0 + w_1x_1 + w_2x_2))},$$

мы можем заменить ее на

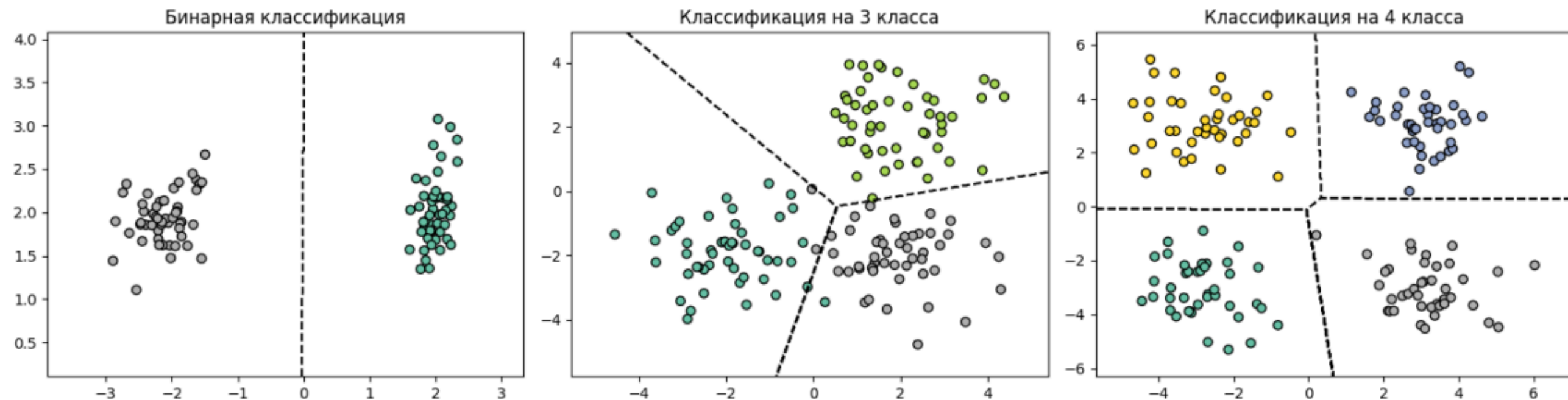
$$\sigma(X, w) = \frac{1}{1 + \exp(-(w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2 + w_5x_1^3 + w_6x_2^3))},$$

Многоклассовая классификация



Многоклассовая классификация

Пусть перед нами стоит задача не бинарной классификации, а многоклассовой, то есть $y \in \{1, \dots, L\}$



Здесь существуют два общепринятых метода отделения классов друг от друга:

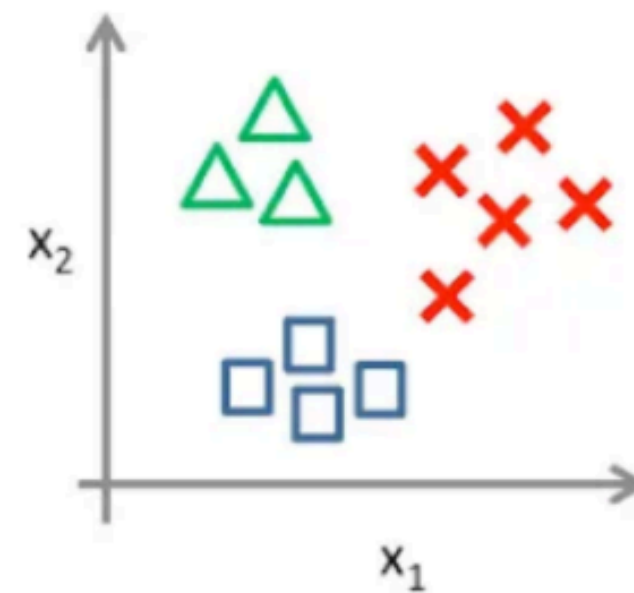
- one-vs-all;
- one-vs-one.



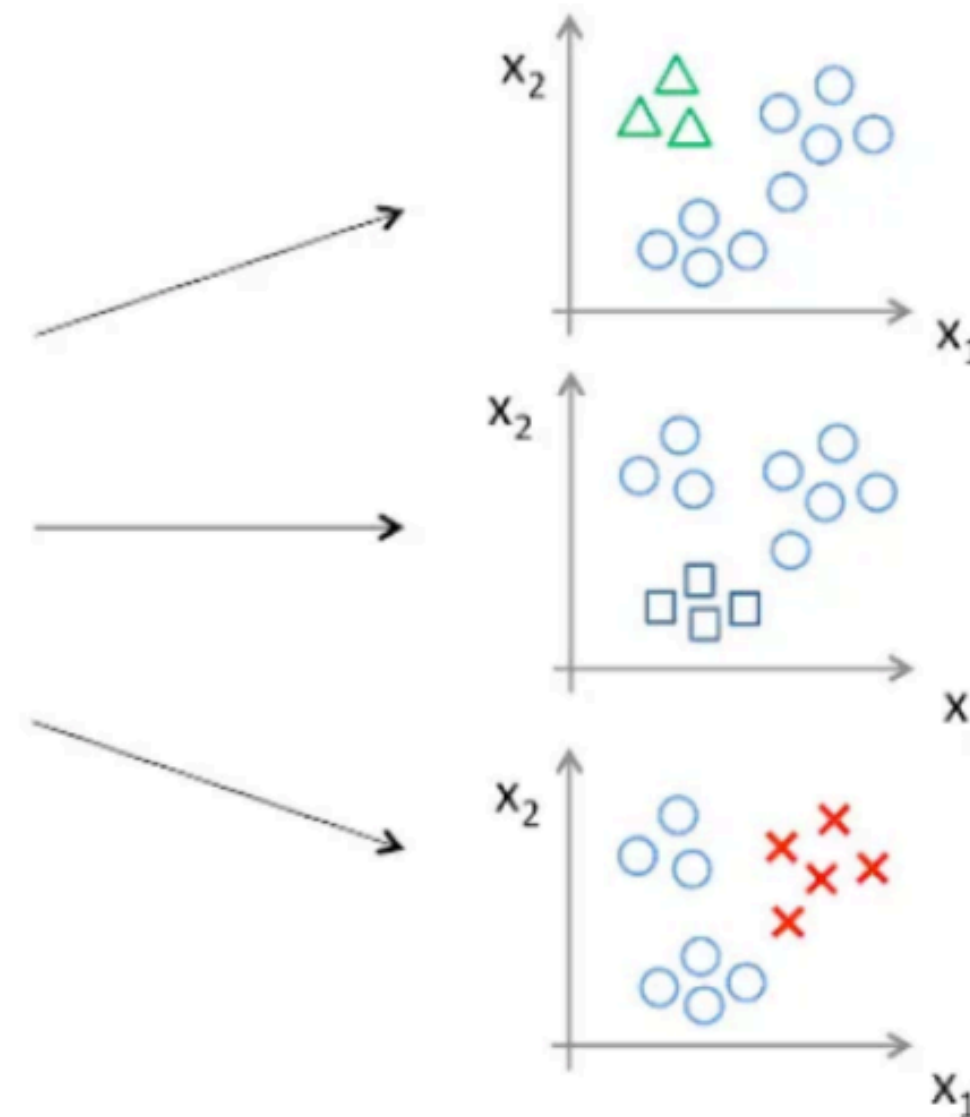
One-vs-All

Сперва рассмотрим метод one-vs-all. Данный метод заключается в том, что мы отдельно выбираем один класс, а все остальные смешиваем в одну кучу. Таким образом, мы строим разделяющую гиперплоскость, которая отделяет один класс от всех остальных. Таким образом, мы последовательно проходим по каждому классу

One-vs-all (one-vs-rest):



Class 1: Green
Class 2: Blue
Class 3: Red





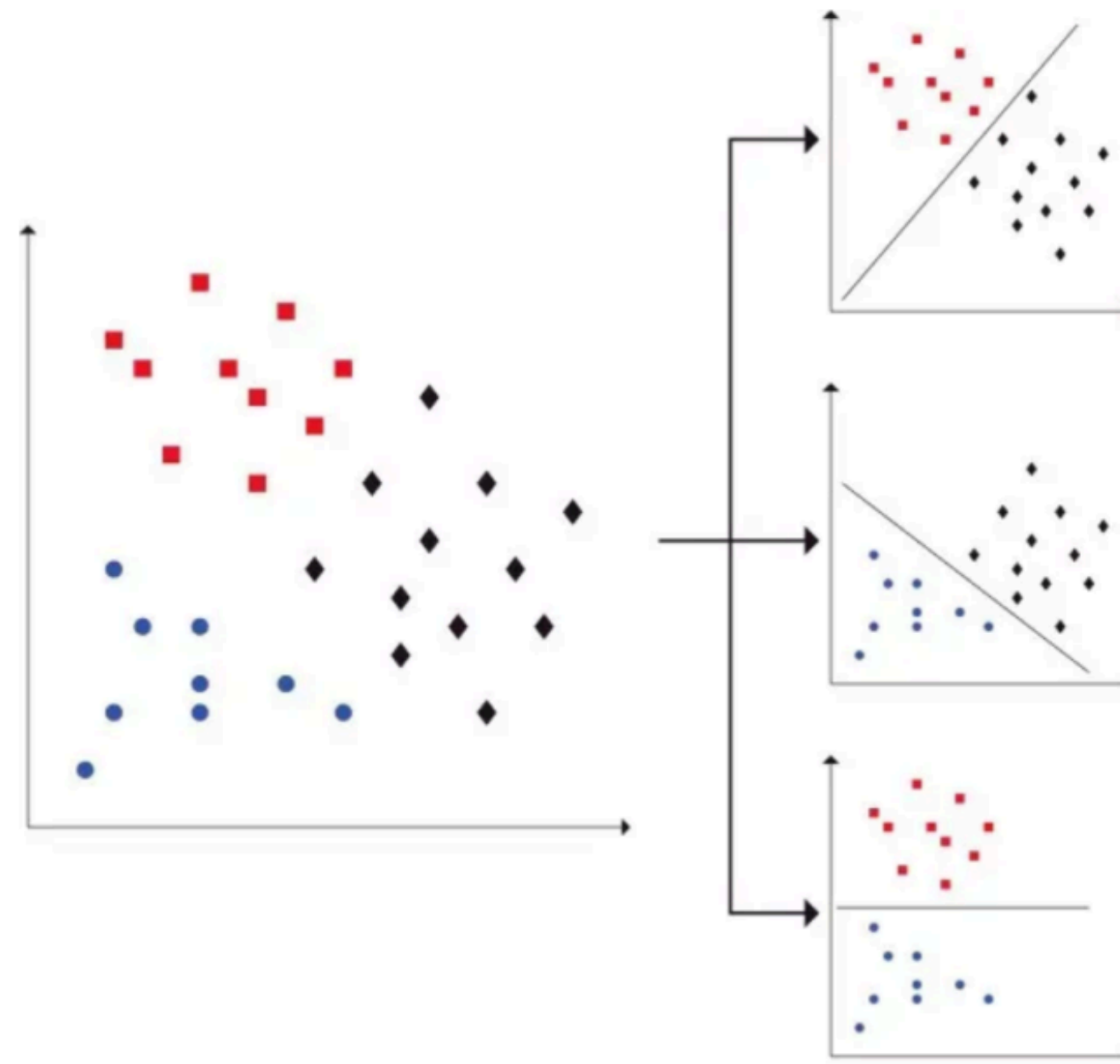
One-vs-All

Очевидно, что в данном случае нам требуется построить L классификаторов. При этом для каждого из классификаторов необходимо подать всю выборку. Следовательно, построение каждого из классификаторов может быть весьма трудоемким процессом, если выборка большого размера.



One-vs-One

Метод one-vs-one заключается в том, что мы выбираем одну пару классов и строим разделяющую гиперплоскость между ними. Таким образом, мы последовательно разделяем все пары классов





One-vs-One

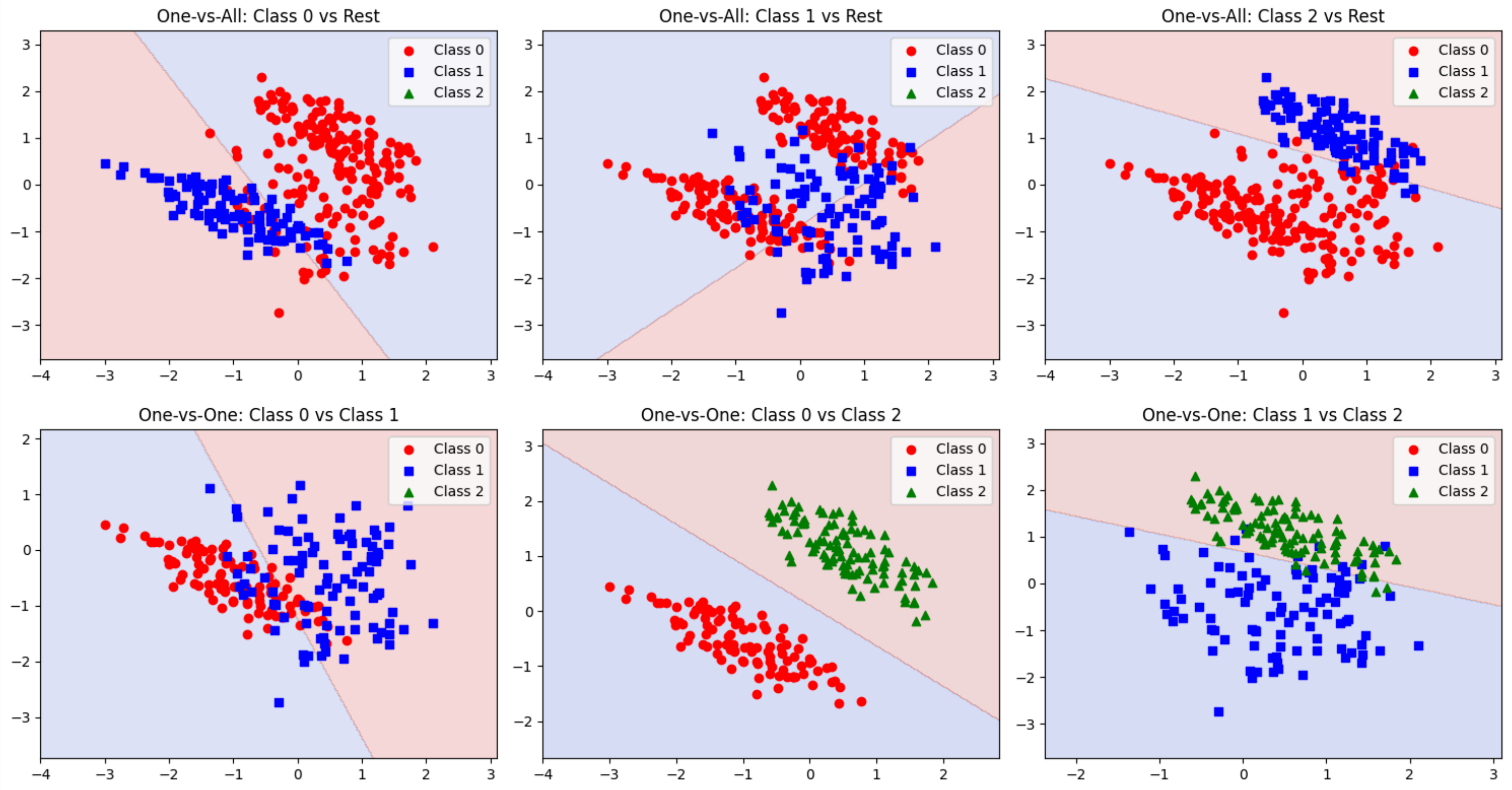
В данном случае требуется построить $\frac{L(L-1)}{2}$ классификаторов. При этом для каждого классификатора необходимо подать только подвыборку, соответствующую двум сравниваемым классам. Следовательно, процесс разделения всех классов может быть достаточно трудоемким, если мы имеем много классов L .

	One-vs-All	One-vs-One
Классификаторы	L	$\frac{L(L-1)}{2}$
Данные	Вся выборка	По подвыборкам

Таблица 4.1: Сравнение стратегий многоклассовой классификации



One-vs-All vs One-vs-One





One-vs-All vs One-vs-One



Оценка качества модели



Confusion matrix

Рассмотрим классические метрики, которые позволяют оценить качество модели классификации.

1. **Матрица ошибок (confusion matrix)** – это таблица, которая позволяет понять, какие ошибки допускает классификатор. Для бинарной классификации она выглядит так:

	Предсказано: +1	Предсказано: -1
Фактически: +1	ТР (True Positive) : верно классифицированные +1	FN (False Negative) : +1, классифицированные как -1
Фактически: -1	ФР (False Positive) : -1, классифицированные как +1	ТН (True Negative) : верно классифицированные -1



Accuracy

2. **Accuracy (точность)** показывает долю правильно предсказанных объектов среди всех наблюдений

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{1}{n} \sum_{i=1}^n [y_i = \hat{y}_i].$$



Precision, Recall

3. **Precision** (точность положительных предсказаний) измеряет, какую долю среди объектов, предсказанных положительными, действительно являются положительными

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Если высокий процент объектов, предсказанных как положительные, оказывается ошибочным, precision будет низкой.

4. **Recall** (полнота) показывает, какую долю от всех реальных положительных объектов модель смогла правильно обнаружить

$$\text{Recall} = \frac{TP}{TP + FN}.$$

Высокий recall означает, что модель не пропускает положительных примеров, что важно, например, в обнаружении болезней.



F1-score

5. **F1-score** – это гармоническое среднее между precision и recall. Оно даёт сбалансированную оценку, если важны оба показателя

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

F1-score особенно полезен, когда необходимо учитывать компромисс между precision и recall.



ROC

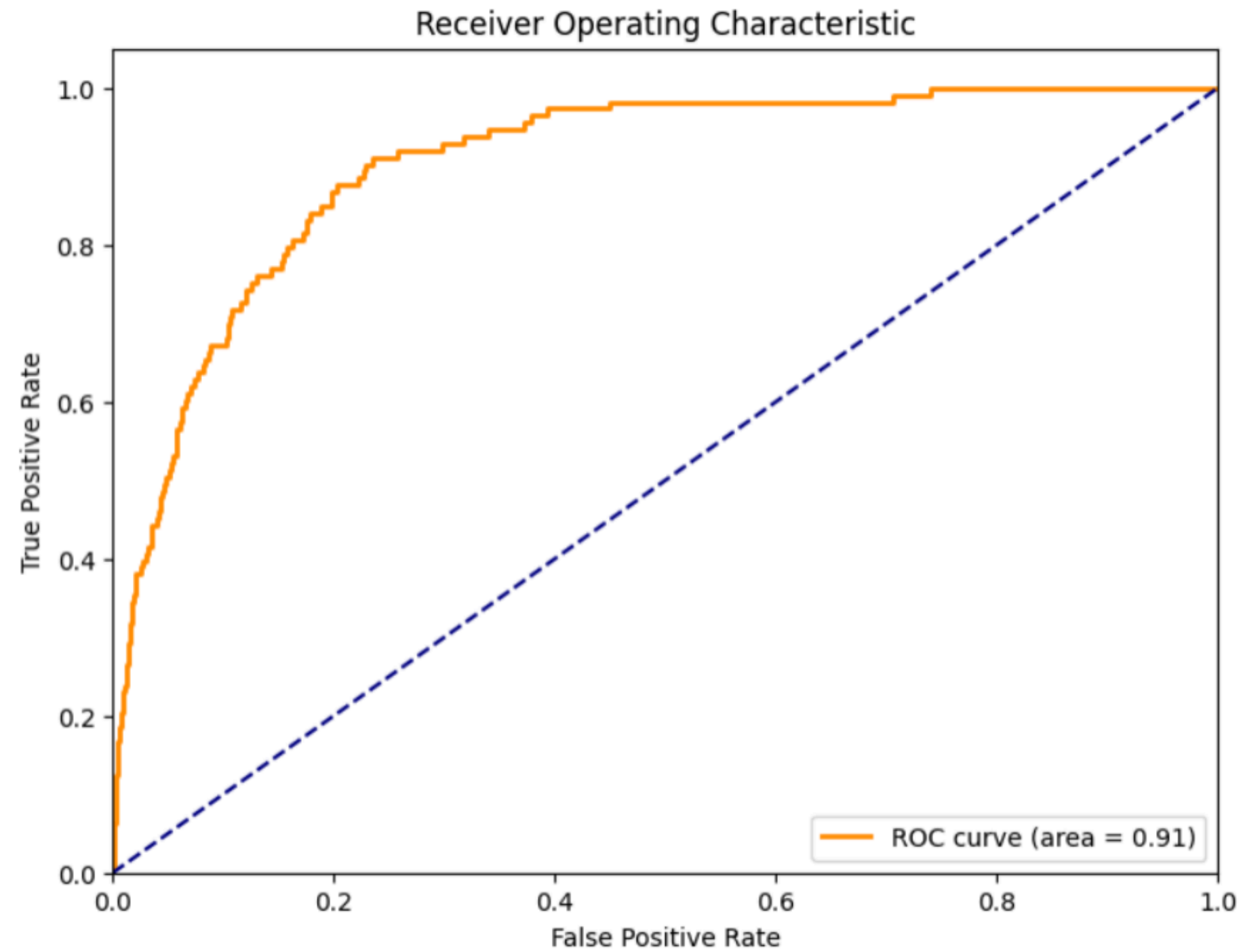
6. **ROC AUC** (Area Under the Receiver Operating Characteristic Curve). **ROC** (Receiver Operating Characteristic) кривая – это график, где по оси X откладывается **FPR** (False Positive Rate), а по оси Y – **TPR** (True Positive Rate, то есть **Recall**). False Positive Rate (FPR) рассчитывается так:

$$\text{FPR} = \frac{FP}{FP + TN}.$$

Он показывает, какую долю отрицательных объектов модель ошибочно классифицировала как положительные.



ROC AUC





ROC AUC

ROC AUC – это площадь под ROC кривой. Это число от 0 до 1:

- Значение 1 означает идеальный классификатор.
- 0.5 соответствует случайному угадыванию.
- Чем выше ROC AUC, тем лучше модель отделяет положительные примеры от отрицательных при различных порогах принятия решения.

Интерпретация: Если для модели $\text{ROC AUC} = 0.9$, это означает, что при случайном выборе одного положительного и одного отрицательного примера с вероятностью 90% модель присвоит положительному более высокий скор, чем отрицательному.